Relatório 5: Comunicação SPI.
Internet das Coisas - Integral
Prof. Gabriel de Freitas Fernandes
2026.2

EQUIPE:
Arthur Peixoto - 170.214
Gabriel Oliva Gurgel - 168.860
Guilherme Samuel dos Santos - 168.867

1 Introdução

Um dos protocolos mais difundidos para comunicação serial é o **Serial Peripheral Interface (SPI)**. Trata-se de uma especificação voltada para uma interface onde ocorre comunicação serial síncrona, destinada principalmente à comunicação de curta distância entre dispositivos. O protocolo permite a transmissão de dados em modo *full duplex* e, em sua configuração tradicional, adota uma arquitetura *master-slave*, na qual um único dispositivo atua como mestre (*master*) e os demais como escravos (*slaves*).

Nesta prática, será realizada uma atividade introdutória de comunicação entre dois microcontroladores ESP32, sendo um configurado como dispositivo mestre e o outro como dispositivo escravo. Além da implementação prática da comunicação utilizando o protocolo SPI, serão revisados conceitos fundamentais relacionados ao funcionamento do protocolo, por meio de um estudo baseado na documentação dos dispositivos e nos conteúdos apresentados em aula.

2 Revisão de conceitos

2.1 Funções relevantes

A biblioteca `SPI.h` já está incluída no ambiente de desenvolvimento, portanto, não foi necessário realizar sua instalação separadamente (ARDUINO, 2023). Aqui estão informações sobre as principais funções relacionadas ao SPI usadas:

2.1.1 Interface SPI nativa

- `SPI.begin()`: inicializa o barramento SPI, configurando os pinos SCK (também conhecido como **Serial Clock**, ou apenas **Clock** - sigla **CLK**), MOSI e SS como saídas (com SCK e MOSI em nível baixo e SS em nível alto) e o pino MISO como entrada. Após chamar essa função, o dispositivo opera como controlador no barramento SPI. Não possui retorno nem parâmetros.

- `SPISettings()`: é utilizado para configurar a porta SPI para um dispositivo SPI específico. Ele combina três parâmetros (velocidade do clock, ordem dos bits e modo de dados) em um único objeto, a proporcionar maior organização e clareza durante a configuração e a etapa inicial de *setup*. Essa função não possui valor de retorno.
  Os parâmetros são:

  - `speedMaximum`: a velocidade máxima de clock em *Hz* (hertz). Por exemplo, podemos usar um `int` de `20000000` para um periférico com especificação de até 20 MHz.
  - `dataOrder`: a ordem dos bits para transferência de dados. Os valores permitidos são `MSBFIRST` (bit mais significativo é enviado por primeiro) e `LSBFIRST` (bit menos significativo é enviado por primeiro).
  - `dataMode`: o modo de dados SPI. os valores permitidos são `SPI_MODE0`; `SPI_MODE1`; `SPI_MODE2`; e `SPI_MODE3`.

- `SPI.beginTransaction()`: configura o barramento SPI com as definições especificadas e assume o controle exclusivo do mesmo. Esta função deve ser chamada antes de se comunicar com um periférico

SPI específico, para garantir a configuração correta e evitar conflitos com outros dispositivos SPI ou interrupções. Por padrão, `SPI.begin()` deve ser chamado antes de utilizar esta função. Não possui retorno e possui um parâmetro chamado `mySettings`, onde é necessário chamar um objeto do tipo `SPISettings()` em que é selecionado as configurações de SPI a serem aplicadas.

- `SPI.transfer()`: realiza uma operação simultânea de envio e recebimento de dados por meio do barramento SPI. Isso ocorre em razão de o SPI ser um protocolo **full-duplex**, no qual a transmissão e a recepção de dados são realizadas simultaneamente durante a comunicação. A função não possui valor de retorno e recebe dois parâmetros: `val` (ou `buffer`), responsável por especificar o dado ou conjunto de dados a ser transmitido, e `size`, que define a quantidade de bytes a serem transferidos. Dessa forma, é possível realizar a transmissão de um único byte ou de um conjunto de bytes, quando fornecido um arranjo (*array*). Por padrão, o parâmetro `size` assume o valor correspondente a 1 byte.

2.1.2 Interface proprietária da Espressif

- `spi_slave_initialize()`: inicializa a interface SPI no modo *slave*, configurando o barramento e os parâmetros específicos do dispositivo. A função recebe quatro parâmetros:
  - `host`: que identifica o periférico SPI a ser utilizado;
  - `bus_config`, que contém as configurações dos pinos e do barramento; `slave_config`, que define as configurações específicas do dispositivo *slave*;
  - `dma_chan`, que determina o canal de DMA (*Direct Memory Access*, onde o próprio periférico se encarregará da leitura e processamento, liberando a CPU de quase todo o processamento antes necessário) utilizado. Neste código, é utilizado o periférico `VSPI_HOST`, os endereços das estruturas `busConfig` e `slaveConfig`, e `SPI_DMA_DISABLED`, indicando que a transferência por DMA não será utilizada. A função retorna um valor do tipo `esp_err_t`, utilizado para indicar se a inicialização foi realizada com sucesso ou se ocorreu algum erro.
- `spi_slave_transmit()`: realiza uma transação SPI no modo *slave*, aguardando que o controlador inicie a comunicação e realize a transferência dos dados. A função recebe três parâmetros:
  - `host`: identifica o periférico SPI utilizado;
  - `trans_desc`: é um ponteiro para uma estrutura do tipo `spi_slave_transaction_t` contendo as informações da transação, como o tamanho dos dados e o *buffer* de recepção;
  - `ticks_to_wait`: define por quanto tempo a função deve aguardar pela realização da transação.

  No código, é utilizado `portMAX_DELAY`, permitindo que o dispositivo aguarde indefinidamente até que o controlador inicie a comunicação. A função retorna um valor do tipo `esp_err_t`, sendo possível verificar, por exemplo, se a operação foi concluída com sucesso por meio da comparação com `ESP_OK`.

2.2 Comparação entre UART e SPI (benefícios e malefícios)

Critério	**Universal Asynchronous Receiver/Transmitter (UART)**	**Serial Peripheral Interface (SPI)**
Tipo de comunicação	Assíncrona (dispositivos podem operar com clocks internos independentes).	Síncrona (precisa de um clock dedicado). Elimina problemas relacionados a desalinhamento de taxa.
Barramento	Apenas dois fios são o suficiente (**TX** e **RX**).	No mínimo, 4 fios (Clock, MOSI, MISO, CS)
Topologia	Ponto a ponto (1 mestre - 1 dispositivo)	Mestre com múltiplos escravos (1 CS por escravo).
Velocidade típica	Baixa a moderada (até aproximadamente 1 Mbps)	Alta (dezenas de Mbps)
Configuração	Taxa de transmissão (conhecido como *baud rate*) deve ser igual em ambos os lados	Apenas configuração de modo de clock (alto ou baixo)
Transmissão	Bits de start/stop e paridade opcional	Mínimo, sem bits extras por byte
Escalabilidade	Difícil escalar para múltiplos dispositivos	Fácil, basta adicionar mais linhas CS
Detecção de erro	Paridade opcional	Nenhuma nativa
Distância de uso	Maior alcance (ex.: RS-232, RS-485)	Curta distância (dentro da placa)
Complexidade	Geralmente é mais simples	Geralmente é mais complexo (ex.: gerenciamento de CS e modos)

Tabela 1: Comparação direta entre comunicação UART e SPI

3 Prática

3.1 Montagem Experimental

O ESP32 dispõe de quatro periféricos de comunicação SPI, denominados **SPI0**, **SPI1**, **HSPI** e **VSPI**, segundo a sua documentação oficial de programação. Para a realização da prática, foi adotada a interface **VSPI**, pois foi a configuração inicialmente definida pelo grupo. Não foi necessário testar outra maneira de ligação.

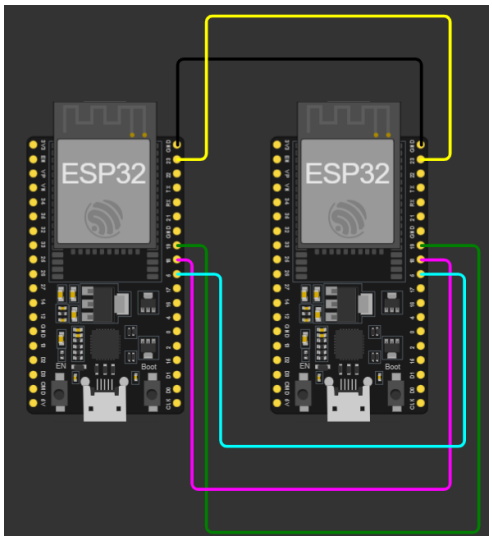


Figura 1: Simulação das entradas

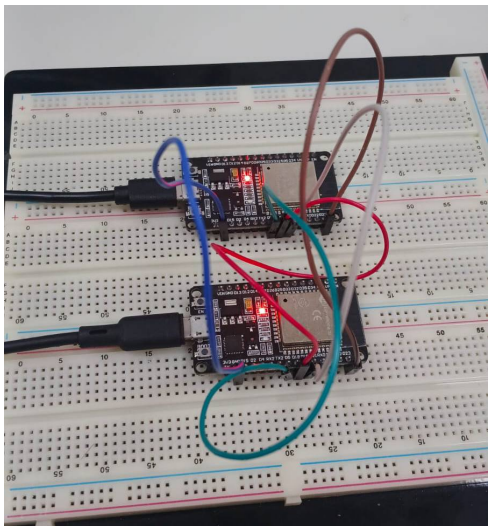


Figura 2: Montagem do circuito feita no laboratório

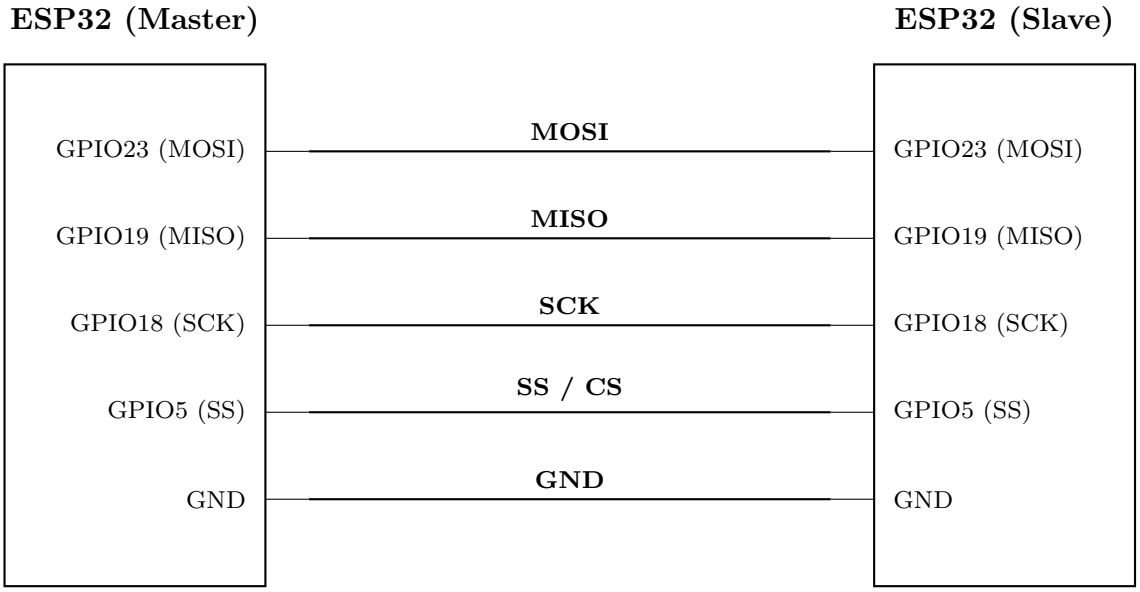


Figura 3: Esquema das entradas

3.2 Código

Para o *master*, chegamos a usar primeiramente um valor de 70 MHz, visto que a interface de SPI do ESP32 consegue transmitir até no máximo, aproximadamente 80 MHz (ESPRESSIF, 2026). Todavia, resolvemos diminuir drasticamente a frequência com o objetivo de evitar saídas inconsistentes no Monitor Serial, além de termos maior certeza que o byte foi transmitido. Assim, para padronização, definimos um valor de 15MHz.

Em relação a comunicação serial USB entre os controladores e o ambiente de desenvolvimento do computador, usamos um *baud rate* de `9600`, por ser o valor que estamos usando nas aulas, apesar da documentação usar `115200`. A convenção de colocar o SS em pino alto para parar a transmissão, e pino baixo para iniciar vem do modo 0 do SPI, definido no último argumento do objeto `SPISettings`.

Código 1: Código do controlador *master*

```
#include <SPI.h>
#define CS_PIN 5

void setup() {
  Serial.begin(9600);
  pinMode(CS_PIN, OUTPUT);
  digitalWrite(CS_PIN, HIGH);
  SPI.begin();
}

void loop() {
  byte dataToSend = 65; // Letra 'A' ou um inteiro 65
  Serial.print("Byte: ");
  Serial.println((char)dataToSend);
  SPI.beginTransaction(SPISettings(1500000, MSBFIRST, SPI_MODE0));
  digitalWrite(CS_PIN, LOW); // Coloque CS em baixo, para começar a transição
  byte response = SPI.transfer(dataToSend);
  digitalWrite(CS_PIN, HIGH); // Coloque CS em alto, para terminar a transição
  SPI.endTransaction();
  delay(1000);
}
```

A implementação do código do *slave* veio inicialmente reutilizamos a mesma lógica empregada no *master*, considerando a característica do protocolo SPI de realizar a comunicação em *full-duplex*, permitindo o envio e o recebimento de dados simultaneamente. Entretanto, essa abordagem de usar o mesmo `SPI.transfer()` não apresentou o funcionamento esperado, uma vez que não foram observados resultados realmente efetivos de transmissão durante o monitoramento da comunicação, apenas bytes aleatórios inconclusivos.

Após novas pesquisas e análises da documentação, optamos pela utilização da `driver/spi_slave.h`, que fornece recursos específicos para a configuração do ESP32 como um *slave* no barramento. Um possível motivo, apontado por participantes do fórum da Espressif, é que a utilização genérica para implementar um dispositivo *slave*, sem os recursos específicos citados, pode levar o dispositivo a tentar gerar seu próprio sinal de *clock*. Como ele deve ser gerado exclusivamente pelo *master*, caso ambos os dispositivos tentem controlar a transmissão de SCK, pode ocorrer um conflito elétrico, uma vez que, segundo os usuários: "essa linha normalmente opera em configuração *push-pull*, podendo resultar em correntes superiores aos limites especificados."(ESPRESSIF, 2026)
Portanto, foi possível implementar a comunicação utilizando diretamente os mecanismos disponibilizados pelo *driver* específico para essa finalidade. Aqui é importante destacar as seguintes *structs* (estruturas):

- `spi_bus_config_t`: estrutura utilizada para definir a configuração dos pinos e características do barramento SPI. No código, os campos `mosi_io_num`, `miso_io_num` e `sclk_io_num` são utilizados para especificar, respectivamente, os pinos MOSI, MISO e SCLK.

- `spi_slave_interface_config_t`: estrutura responsável por armazenar as configurações específicas da interface SPI no modo *slave*. O campo `spics_io_num` define o pino utilizado pelo sinal CS (*Chip*

Select), enquanto `mode` determina o modo SPI utilizado na comunicação. Neste código, o valor `0` corresponde ao `SPI_MODE0`. O campo `queue_size` determina a quantidade máxima de transações que podem ser mantidas na fila, sendo configurado como `1` (um byte, não estamos enviando um *array* em um *buffer*).

- `spi_slave_transaction_t`: estrutura utilizada para descrever uma transação SPI realizada pelo dispositivo *slave*. No código, o campo `length` especifica o tamanho da transferência em bits, sendo configurado como `8`, correspondente a um byte. O campo `rx_buffer` recebe o endereço da variável `data`, indicando o local da memória no qual os dados recebidos pelo *slave* serão armazenados. Dessa forma, após a conclusão da transação, o byte recebido pelo dispositivo pode ser acessado por meio da variável `data`.

Código 2: Código do controlador *slave*

```
#include "driver/spi_slave.h"

#define PIN_SCLK 18
#define PIN_MISO 19
#define PIN_MOSI 23
#define PIN_CS 5
#define SPI_HOST VSPI_HOST

byte data;

void setup() {
    Serial.begin(9600);
    spi_bus_config_t busConfig = {};
    busConfig.mosi_io_num = PIN_MOSI;
    busConfig.miso_io_num = PIN_MISO;
    busConfig.sclk_io_num = PIN_SCLK;
    spi_slave_interface_config_t slaveConfig = {};
    slaveConfig.spics_io_num = PIN_CS;
    slaveConfig.mode = 0;
    slaveConfig.queue_size = 1;
    esp_err_t result = spi_slave_initialize(SPI_HOST, &busConfig, &slaveConfig, SPI_DMA_DISABLED);
}

void loop() {
    data = 0;
    spi_slave_transaction_t transaction = {};
    transaction.length = 8;
    transaction.rx_buffer = &data;
    esp_err_t result = spi_slave_transmit(SPI_HOST, &transaction, portMAX_DELAY);
    if (result == ESP_OK) Serial.printf("Received: %c\n", (char)data);
}
```

3.3 Resultados e Discussão

Foi possível verificar que a transmissão inicialmente realizada utilizava bytes fixos, como o `char 'A'` ou o `int 65`.

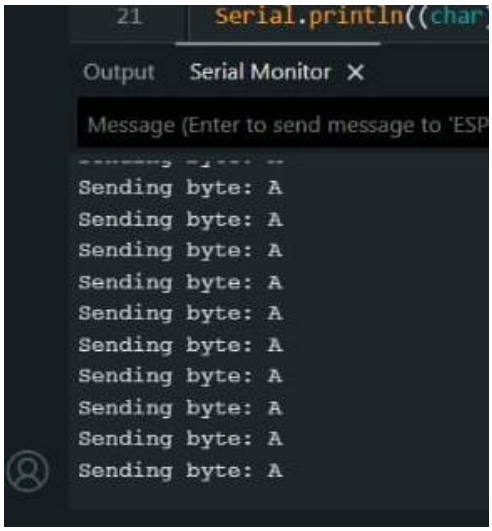


Figura 4: Monitoramento serial no *master*

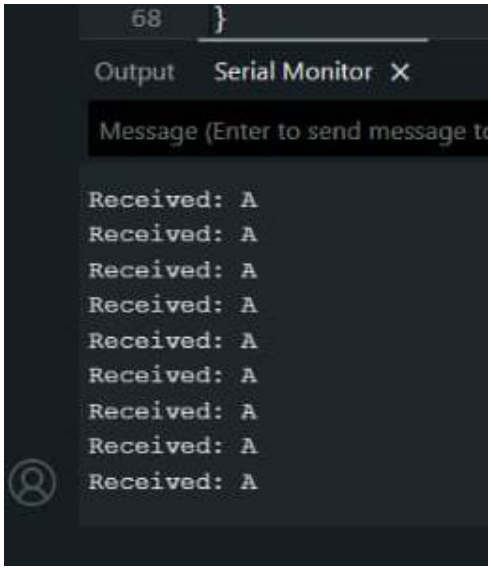


Figura 5: Monitoramento serial no *slave*

Para realizar testes mais abrangentes, optamos por utilizar a função `Serial.read()` no código-fonte do *master*, permitindo verificar a transmissão de diferentes bytes e de *strings*, ao digitar pelo teclado do computador. Os testes também foram realizados com sucesso, confirmando o funcionamento da comunicação para diferentes tipos de dados.

3.4 Verificação com Osciloscópio



Figura 6: Estado do barramento CS visto em detalhes

Por meio do monitoramento com o osciloscópio, foi possível observar um pouco do comportamento do sinal de CS, e seu comportamento durante a transmissão. Na Figura 6, o pino CS permanece em nível baixo enquanto o byte é transmitido, intervalo que corresponde a cerca de 8 pulsos de clock, um para cada bit. A subida do sinal para nível alto indica o término da transmissão, após o qual o algoritmo do mestre aguarda 1 segundo (*delay*) antes do envio seguinte.

3.4.1 Comunicação entre ESP32 e outros dispositivos

Agora que foi possível estabelecer a comunicação entre dois ESP32, podemos analisar algumas características relacionadas à quantidade de dispositivos simultâneos no SPI. Caso seja necessário conectar vários ao mesmo controlador, as linhas MOSI, MISO e SCK podem ser compartilhadas entre todos os dispositivos, enquanto cada dispositivo deve possuir uma forma individual de seleção por meio do CS. Durante uma transferência de bytes, somente o dispositivo selecionado deve habilitar sua saída na linha MISO, enquanto os demais permanecem desabilitados. No caso da configuração utilizada neste relatório, o dispositivo é selecionado quando o sinal CS é colocado em nível baixo.

Essa característica faz com que a quantidade de dispositivos conectados diretamente ao barramento seja limitada principalmente pela quantidade de GPIOs disponíveis para os sinais CS. Entretanto, esse limite pode ser contornado com o uso de circuitos adicionais, como um decodificador `74HC138`. Para uma quantidade muito grande de dispositivos ou quando o circuito exigir maior distância entre eles, também pode ser mais adequado utilizar outro protocolo de comunicação.

Também é importante considerar a compatibilidade dos níveis lógicos entre os dispositivos conectados. Caso um microcontrolador utilize uma tensão lógica diferente daquela suportada pelo ESP32, deve ser utilizado um conversor de nível lógico (*logic level converter*) apropriado para os sinais digitais, garantindo que as tensões presentes nas linhas SCK, MOSI, MISO e CS estejam dentro dos níveis aceitáveis para todos os dispositivos envolvidos.

Embora o protocolo SPI não estabeleça, por si só, um número máximo universal de dispositivos conectados ao barramento, existem limitações práticas relacionadas ao *hardware* utilizado.Os principais fatores são:

- **Quantidade de GPIOs disponíveis**: quando cada dispositivo utiliza um sinal CS dedicado, a quantidade de GPIOs disponíveis no controlador pode se tornar uma das limitações mais imediatas.
- **Capacidade do barramento**: a conexão de diversos dispositivos em paralelo aumenta a capacitância total das linhas SCK, MOSI e MISO. Essa maior carga capacitiva pode aumentar os tempos de subida e descida dos sinais e prejudicar sua integridade, principalmente em frequências elevadas, podendo exigir a redução da frequência do clock.
- **Comprimento dos fios**: cabos longos e conexões com múltiplos dispositivos podem provocar reflexões, interferências eletromagnéticas e degradação da integridade do sinal. Dessa forma, quanto maior o comprimento e a complexidade da interconexão, maior a necessidade de atenção ao roteamento, aterramento e, dependendo da aplicação, à terminação dos sinais.
- **Capacidade elétrica dos GPIOs**: além da quantidade de dispositivos, deve-se respeitar os limites de corrente e as características elétricas dos GPIOs do controlador. O número de entradas conectadas e demais componentes associados às linhas podem influenciar a capacidade de acionamento dos sinais, gerando lixo na transmissão.

Portanto, embora não exista um limite de dispositivos definido diretamente pelo protocolo SPI, a escalabilidade do barramento é determinada, na prática, pela combinação entre a quantidade de sinais CS disponíveis, a capacidade elétrica do controlador, a frequência de operação, a capacitância e o comprimento das interconexões e a integridade dos sinais. À medida que o número de dispositivos aumenta, torna-se necessário avaliar esses fatores para garantir uma comunicação confiável.

4 Dificuldades Encontradas

A maior dificuldade do grupo foi conectar o ESP32 ao ambiente de desenvolvimento via USB. Mesmo alternando as portas e testando em outro computador, não era possível confirmar se a saída do Monitor Serial continha *bytes* legítimos ou apenas ruído. O problema só foi solucionado após a reinstalação do *driver* da Espressif.

5 Anexos

- Dois arquivos `.ino`, correspondentes aos códigos dos controladores *master* e *slave*, respectivamente.

Referências

ARDUINO. *Arduino with SPI Communication*. 2023. Disponível em: ⟨https://docs.arduino.cc/learn/communication/spi⟩.

ESPRESSIF. *Serial Peripheral Interface (SPI)*. 2026. Disponível em: ⟨https://docs.espressif.com/projects/esp-faq/en/latest/software-framework/peripherals/spi.html⟩.

ESPRESSIF. *SPI Slave Driver*. 2026. Disponível em: ⟨https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/spi_slave.html⟩.